Harry Potter API paths (`main.py`)

This notebook calls the **`/harry-potter`** routes from **`app/main.py`** over HTTP using the **`requests`** package.

Prerequisite: Run the API from the repository root, for example **`uvicorn app.main:app --reload --port 8000`**. The default base URL is **`http://127.0.0.1:8000`**; override with the environment variable **`API_BASE_URL`** if you use another host or port.

Note: Cells that **`POST`**, **`PUT`**, or **`DELETE`** modify **`data/harry_potter_data.json`**. Run create/update/delete cells once, or restore the file from git if you repeat them.

In [10]:
```python
import os

import requests

BASE_URL = os.environ.get("API_BASE_URL", "http://127.0.0.1:8000").rstrip("/")

try:
    _health = requests.get(f"{BASE_URL}/health", timeout=5)
    _health.raise_for_status()
except requests.RequestException as exc:
    raise RuntimeError(
        f"Cannot reach API at {BASE_URL!r}. From the repo root run e.g. "
        "`uvicorn app.main:app --reload --port 8000`, then rerun this cell "
        "(or set API_BASE_URL if the server uses another host/port)."
    ) from exc

# Sample id from bundled JSON (Harry Potter)
SAMPLE_CHARACTER_ID = "103cb127-8bb3-4a34-9fd3-c193a9a6cf54"
```

`GET /harry-potter`

List/search with optional query parameters: `first_name`, `last_name`, `house_name` (equality template).

In [11]:
```
resp = requests.get(f"{BASE_URL}/harry-potter", timeout=30)
assert resp.status_code == 200, resp.text
data = resp.json()
assert "items" in data
print("count:", len(data["items"]))
data["items"][:3]
```

```
count: 20
```

Out[11]:
```
[{'id': '103cb127-8bb3-4a34-9fd3-c193a9a6cf54',
  'first_name': 'Harry',
  'last_name': 'Potter',
  'house_name': 'Gryffindor'},
 {'id': '3b3cdd9b-1763-46c0-9731-9c99f7ebfcf9',
  'first_name': 'Hermione',
  'last_name': 'Granger',
  'house_name': 'Gryffindor'},
 {'id': '778fc3ef-1fce-4f41-a83e-959de33e77f0',
  'first_name': 'Ronald',
  'last_name': 'Weasley',
  'house_name': 'Gryffindor'}]
```

In [12]:
```
resp = requests.get(
    f"{BASE_URL}/harry-potter", params={"house_name": "Gryffindor"}, timeout=30
)
assert resp.status_code == 200
items = resp.json()["items"]
assert all(r["house_name"] == "Gryffindor" for r in items)
len(items)
```

Out[12]: 7

GET /harry-potter/{character_id}

Returns **404** if the character does not exist.

In [13]:
```
resp = requests.get(f"{BASE_URL}/harry-potter/{SAMPLE_CHARACTER_ID}", timeout=30)
assert resp.status_code == 200
resp.json()
```

Out[13]:
```
{'id': '103cb127-8bb3-4a34-9fd3-c193a9a6cf54',
 'first_name': 'Harry',
 'last_name': 'Potter',
 'house_name': 'Gryffindor'}
```

In [14]:
```
missing = requests.get(
    f"{BASE_URL}/harry-potter/00000000-0000-0000-0000-000000000000", timeout=30
)
assert missing.status_code == 404
missing.json()
```

Out[14]:
```
{'detail': "No character with id '00000000-0000-0000-0000-000000000000'"}
```

POST /harry-potter

Creates a character; server assigns **id** if omitted. Response body is the new id (JSON string).

In [15]:
```
payload = {
    "first_name": "Notebook",
    "last_name": "TestCharacter",
    "house_name": "Ravenclaw",
}
resp = requests.post(f"{BASE_URL}/harry-potter", json=payload, timeout=30)
assert resp.status_code == 200, resp.text
new_id = resp.json()
assert isinstance(new_id, str)
new_id
```

Out[15]:
```
'c865d550-f9a2-4351-a4d7-b1dda2197acd'
```

PUT /harry-potter/{character_id}

Updates by id; **400** if the character does not exist.

In [16]:
```
# Uses `new_id` from the POST cell above — run that cell first.
update_body = {
    "first_name": "Notebook",
```

```
    "last_name": "TestCharacter",
    "house_name": "Hufflepuff",
}
resp = requests.put(f"{BASE_URL}/harry-potter/{new_id}", json=update_body, timeout=30)
assert resp.status_code == 200
assert resp.json() == {"updated": 1}
requests.get(f"{BASE_URL}/harry-potter/{new_id}", timeout=30).json()
```

Out[16]:
```
{'id': 'c865d550-f9a2-4351-a4d7-b1dda2197acd',
 'first_name': 'Notebook',
 'last_name': 'TestCharacter',
 'house_name': 'Hufflepuff'}
```

DELETE /harry-potter/{character_id}

Returns `{"deleted": 0}` or `{"deleted": 1}`.

In [17]:
```
resp = requests.delete(f"{BASE_URL}/harry-potter/{new_id}", timeout=30)
assert resp.status_code == 200
assert resp.json()["deleted"] == 1

gone = requests.get(f"{BASE_URL}/harry-potter/{new_id}", timeout=30)
assert gone.status_code == 404
gone.json()
```

Out[17]:
```
{'detail': "No character with id 'c865d550-f9a2-4351-a4d7-b1dda2197acd'"}
```

In [ ]:
```

```